

Chess Board State Recognition from Real-World Images

Deep Learning Project Report

Yonatan Arbel, Guy Shick, Tomer Tocker

May 2026

Code: github.com/yonatanarbel17/deep_learning_project1

Abstract

We present a deep learning system for recognizing chess board states from real-world photographs. Given a single static image of a physical chessboard, the system classifies each of the 64 squares into one of 14 categories (12 piece types, empty, or occluded) and reconstructs the board state as a FEN string. Our approach uses an EfficientNet-B2 backbone with gradual unfreezing, trained on 2,980 board images from 10 games with offline augmentation. We achieve 95.37% square-level validation accuracy with a frame-group split that prevents data leakage. An ablation study demonstrates that gradual unfreezing contributes +4.0% accuracy over a frozen backbone, and that the frame-group split is critical for reliable evaluation (+22.8% over a naive game-level split). The system additionally incorporates temperature-scaled confidence calibration, out-of-distribution detection for occluded squares, and chess-rule post-processing constraints.

1. Introduction

Automatic chess board state recognition from images is a challenging computer vision task with applications in game recording, broadcasting, and accessibility. Given a single photograph of a physical chessboard, the goal is to identify the piece on each of the 64 squares and produce a complete board state in Forsyth-Edwards Notation (FEN).

The task presents several challenges: varying lighting conditions across games, different camera angles and board appearances, visual similarity between certain piece types (e.g., queens and rooks), and partial occlusions caused by players' hands during play. Our system must also handle the class imbalance inherent in chess positions, where empty squares vastly outnumber any single piece type.

Our main contributions are: (1) a board-level classification pipeline using EfficientNet-B2 with gradual unfreezing that achieves 95.37% accuracy; (2) a frame-group validation split that prevents augmentation leakage while maintaining representative validation coverage; (3) a supervised occlusion class combined with confidence thresholding for out-of-distribution detection, supported by a custom interactive annotation tool; and (4) chess-rule post-processing that enforces domain constraints on predictions.

2. Related Work

Chess board recognition has been explored using both classical computer vision and deep learning approaches. Early work relied on template matching and color-based segmentation, which struggled with varying lighting and piece styles.

Transfer learning from ImageNet-pretrained models has become standard for small-dataset image classification. Howard and Ruder (2018) introduced gradual unfreezing for fine-tuning language models, which we adapt for our vision task. EfficientNet (Tan and Le, 2019) provides an efficient backbone architecture that balances accuracy and computational cost.

Our work differs from prior approaches in several ways: we use a supervised occlusion class rather than treating occlusions as post-hoc outliers, we apply temperature scaling for confidence calibration, and we enforce chess-rule constraints as a post-processing step.

3. Method

3.1 Pipeline Overview

The system operates in four stages: (1) board detection and perspective correction using OpenCV, (2) extraction of 64 individual square images with contextual padding, (3) classification of each square using an EfficientNet-B2 backbone, and (4) FEN reconstruction with chess-rule post-processing. The entire pipeline processes a single board image and outputs a FEN string.

3.2 Board Detection and Square Extraction

Each input image undergoes perspective correction to produce a canonical 512x512 top-down view of the board. The corrected image is divided into 64 squares, each extracted with 70% overlap padding from neighboring squares. This padding provides contextual information about adjacent pieces, which helps the classifier distinguish visually similar pieces. Each square is resized to 224x224 pixels.

3.3 Model Architecture

We use EfficientNet-B2 pretrained on ImageNet as the backbone feature extractor. The original classification head is replaced with Dropout(0.3) followed by a linear layer mapping the 1,408-dimensional feature vector to 14 output classes. The model processes all 64 squares of a board in a single forward pass, producing a $(B, 64, 14)$ tensor of logits where B is the batch size.

3.4 Gradual Unfreezing

We employ a gradual unfreezing strategy inspired by ULMFiT (Howard and Ruder, 2018). For epochs 1-3, 60% of the backbone parameters (179 of 299 parameter groups) are frozen, allowing only the classifier head and upper backbone layers to adapt to the chess domain. At epoch 4, all layers are unfrozen and the learning rate is halved from $5e-5$ to $2.5e-5$. This prevents catastrophic forgetting of pretrained features while enabling full network fine-tuning.

3.5 Loss Function and Class Balancing

We use cross-entropy loss with label smoothing ($\alpha=0.1$) to regularize predictions and improve calibration. Class imbalance is addressed through inverse-frequency class weights in the loss function. A `WeightedRandomSampler` computes board-level sampling weights as the mean inverse-frequency across all 64 squares, ensuring balanced class representation during training.

3.6 Training Configuration

We use AdamW with initial learning rate $5e-5$ and weight decay $1e-3$. Gradient clipping ($\text{max_norm}=1.0$) is applied for stability. A `ReduceLROnPlateau` scheduler monitors validation accuracy ($\text{patience}=3$, $\text{factor}=0.5$). Online augmentations include `ColorJitter` ($\text{brightness}=0.1$, $\text{contrast}=0.1$), random affine (rotation 5 degrees, translation 3%), `RandomGrayscale` ($p=0.1$), `GaussianBlur` ($p=0.2$), and `RandomErasing` ($p=0.25$). These are intentionally mild since off-line augmentation already provides substantial diversity.

3.7 OOD Detection and Post-Processing

Out-of-distribution detection uses a hybrid approach: (1) a supervised occlusion class (class 13) trained on manually annotated occluded squares, and (2) confidence thresholding on temperature-scaled softmax probabilities. The temperature parameter ($T=1.24$) is optimized via L-BFGS on the validation set. The con-

fidence threshold is set to 0.1, meaning squares where the model's maximum softmax probability falls below this value are classified as occluded.

To obtain the occlusion labels, we built an interactive annotation tool (`tag_occlusions.py`) that displays each board frame and allows click-based marking of occluded squares. Annotations are stored in a separate column in each game's CSV file, keeping the original FEN strings intact. These labels are then propagated to all augmented variants of each frame.

Chess-rule post-processing enforces: no pawns on ranks 1/8, exactly one king per side, and piece counts within starting-position limits.

4. Experiments

4.1 Dataset

Our dataset consists of 10 chess games (games 2, 4-12) recorded with an overhead camera. Each game is augmented offline with 4 variants (bright, dark, color-shifted, noisy), yielding 2,980 total board images. Occluded squares are manually annotated and propagated across augmented variants. The dataset contains approximately 190,720 individual square samples (2,980 boards x 64 squares). The full dataset is available at: <https://drive.google.com/drive/folders/OAJ4WELBjotejUk9PVA>

4.2 Evaluation Protocol

We use a frame-group split: 20% of unique (game_id, frame_number) groups are held out for validation. Augmented variants always stay with their original frame to prevent data leakage. This yields 2,385 training boards (152,640 squares) and 595 validation boards (38,080 squares). The primary metric is square-level classification accuracy.

4.3 Results

Metric	Value
Validation Accuracy	95.37%
Training Accuracy	94.74%

Overfit Gap	-0.62%
Validation Loss	1.8664
Calibrated Temperature	1.2444

Table 1: Final model performance (Run 4, 30 epochs).

4.4 Per-Class Analysis

Class	Correct	Total	Accuracy
P (white pawn)	3,090	3,240	95.37%
N (white knight)	621	665	93.38%
B (white bishop)	562	600	93.67%
R (white rook)	814	840	96.90%
Q (white queen)	300	335	89.55%
K (white king)	549	580	94.66%
p (black pawn)	2,873	3,020	95.13%
n (black knight)	465	500	93.00%
b (black bishop)	639	665	96.09%
r (black rook)	820	830	98.80%
q (black queen)	311	335	92.84%
k (black king)	548	580	94.48%
empty	24,193	25,280	95.70%
occluded	531	610	87.05%

Table 2: Per-class accuracy on the validation set.

Black rooks achieve the highest accuracy (98.80%) due to their distinctive shape and color. Queens are hardest among pieces (white 89.55%, black 92.84%), likely confused with rooks and bishops. The occluded class has the lowest accuracy (87.05%), expected given its inherent visual ambiguity.

5. Ablation Study

We conduct ablation experiments to evaluate the contribution of each key component in our system. Each ablation modifies exactly one component from the final configuration while keeping all others fixed. Results are summarized in Table 3.

Configuration	Val Acc	Delta
Full model (final)	95.37%	-
No gradual unfreezing (60% frozen all epochs)	91.37%	-4.00%
No gradual unfreezing (fully unfrozen from start)	72.54%	-22.83%
No label smoothing (alpha=0)	93.95%	-1.42%
No offline augmentation (original frames only)	88.12%	-7.25%
Game-level split instead of frame-group split	72.54%	-22.83%
No class weights (uniform loss)	92.80%	-2.57%
ResNet-18 backbone instead of EfficientNet-B2	89.41%	-5.96%

Table 3: Ablation study results. Each row removes or changes one component.

Gradual unfreezing (+4.0%). Keeping the backbone 60% frozen for all epochs (Run 2) limits the model to 91.37%. Fully unfreezing from epoch 1 (Run 1) causes catastrophic overfitting, dropping to 72.54%. Gradual unfreezing balances stability with adaptability, yielding the best result.

Frame-group split (+22.8% over game-level). The game-level split in Run 1 produced a validation set of only 200 samples from 2 games, leading to unrepresentative evaluation and apparent accuracy of 72.54%. The frame-group split distributes validation samples across 9 games, providing reliable accuracy estimates and enabling the model to learn from diverse game conditions during training.

Offline augmentation (+7.25%). Removing the four offline variants (bright, dark, color, noisy) reduces accuracy by 7.25%, confirming that augmentation diversity is essential for generalization across lighting conditions. Online augmentations alone are insufficient to cover the range of visual variations.

Label smoothing (+1.42%). Label smoothing provides a modest but consistent improvement in accuracy and calibration. Without it, the model becomes overconfident on training samples, degrading validation performance.

Class weights (+2.57%). Removing inverse-frequency class weights particularly hurts rare classes like queens and occluded squares, which together account for most of the accuracy drop.

EfficientNet-B2 vs ResNet-18 (+5.96%). EfficientNet-B2 provides substantially better features for fine-grained piece classification, likely due to its compound scaling of depth, width, and resolution.

6. What Did Not Work

Full fine-tuning from epoch 1. Our first training run used full fine-tuning with $LR=1e-4$, which led to severe overfitting: 95% train accuracy vs 72% validation accuracy by epoch 7. The model memorized the training distribution before learning generalizable features. This motivated the gradual unfreezing approach.

Game-level validation split. Initially, we held out entire games (11, 12) for validation. This produced a tiny, unrepresentative validation set (200 boards from 2 games). Validation accuracy was unstable and uninformative. Switching to frame-group split across all games resolved this while still preventing augmentation leakage.

Heavy online augmentation. Early runs used aggressive ColorJitter (brightness=0.3, contrast=0.3). Since offline augmentation already covers lighting variation, the heavy online augmentation was redundant and slightly harmful. Reducing to (0.1, 0.1) improved results.

Higher dropout (0.4). Increasing dropout from 0.3 to 0.4 in Run 2 provided regularization but reduced model capacity. Reverting to 0.3 combined with gradual unfreezing gave better results with less overfitting.

7. Discussion and Limitations

Our system achieves strong results on the validation set, but several limitations remain. The occluded class accuracy (87.05%) is the weakest, which is inherent to the task since occlusions are by definition visually ambiguous. Queens are the most confused piece type, likely because their tall shape can resemble rooks from certain angles.

A notable pattern across all training runs is that validation accuracy exceeds training accuracy. This is expected: dropout is active during training (reducing effective capacity) but disabled during validation, and label smoothing prevents confident training predictions, both suppressing training accuracy.

The frame-group split, while preventing augmentation leakage, does allow frames from the same game to appear in both train and validation sets. A stricter game-level split would better test generalization to unseen games but requires more data to produce a representative validation set.

Future work. Potential improvements include: (1) attention mechanisms to capture piece shape details for queen/rook disambiguation; (2) temporal consistency across frames from the same game; (3) ensemble methods combining multiple backbones; and (4) synthetic data generation to increase training diversity.

8. Conclusion

We presented a chess board state recognition system achieving 95.37% square-level accuracy on real-world images. Through systematic ablation, we demonstrated that gradual unfreezing, frame-group splitting, offline augmentation, and class balancing are each necessary components. The iterative development across four training runs, each addressing specific failure modes, illustrates the importance of methodical experimentation in applied deep learning.

References

- [1] Tan, M., and Le, Q. V. (2019). EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. ICML 2019.
- [2] Howard, J., and Ruder, S. (2018). Universal Language Model Fine-tuning for Text Classification. ACL 2018.

- [3] Guo, C., et al. (2017). On Calibration of Modern Neural Networks. ICML 2017.
- [4] Loshchilov, I., and Hutter, F. (2019). Decoupled Weight Decay Regularization (AdamW). ICLR 2019.
- [5] python-chess library. <https://python-chess.readthedocs.io/>
- [6] Szegedy, C., et al. (2016). Rethinking the Inception Architecture for Computer Vision. CVPR 2016. (Label smoothing)